

AWS IoT Workshop Request: Day-2 Edge Fleet Operations

BLUF

We are requesting a hands-on AWS workshop focused on using AWS IoT Core capabilities to support Day-2 operations for our industrial edge fleet. The goal is not a generic IoT overview. We want to work with AWS to validate whether AWS IoT Jobs, AWS IoT Device Client, device shadows, fleet indexing, and related patterns can help us move from one-device-at-a-time and script-based operations toward safe, observable, fleet-level management.

Current Context

We operate an IIoT Data Platform with an edge data gateway deployed on industrial edge computers running Debian. Each gateway runs our **edged** daemon, Docker as the edge container runtime, and a typical three-container application stack: a message bus, our I/O gateway product that contains most of the business logic, and **system config**, which acts as an edge-based API.

The edge architecture uses two primary data planes. Telemetry is exported from the edge into our cloud environment for consumption by other applications and services. Device control flows through our control-plane connection, terminating through **edged** and then reaching **system config**. Both paths use MQTTS over mTLS.

Our security model is intentionally restrictive. We do not open static inbound ports or depend on static IP addresses at the edge. All communication is initiated outbound by the edge device, and firewall posture is managed locally using IPv4 and IPv6 iptables rules.

Operational Problem

We are approaching 1,000 connected devices, with forecast to double every year for the foreseeable future. Today, Edge Manager can connect to a device through our **edged** control plane, but it manages devices one at a time. We do not yet have a first-class concept of a fleet or cohort, device shadow, desired state, reported state, or a reconciliation model that can move a subset of devices toward a defined target state.

For Day-1 provisioning, we have standardized on Ansible. It works well for taking a freshly imaged Debian device from vanilla OS to deploy-ready, and we have idempotent roles and playbooks that express the desired host and application state. However, using Ansible for Day-2 production operations has been difficult because standard Ansible assumes reachability to managed hosts, which conflicts with our outbound-only security model. Running Ansible directly on the edge host adds operational complexity and raises the minimum hardware requirement because the device needs enough CPU and memory headroom to execute playbooks locally.

As an interim measure, we built a home-grown application that calls the central Edge Manager API and performs batched operations through our existing control-plane connection. This lets us deploy Max Edge application updates and system utility or configuration updates today, but it is a stopgap. Most

operations are Bash-script driven, which limits idempotency, rollback, state awareness, staged rollout control, and operational reporting.

Near-Term Request

We are asking AWS to run a hands-on workshop using our existing lab devices and realistic Day-2 operations scenarios. The workshop should be practical, technical, and limited to roughly four hours.

The workshop should help us answer one primary question:

Can AWS IoT Core technologies provide a better operating model for Day-2 fleet operations while preserving our outbound-only, mTLS-based industrial edge security posture?

Proposed Workshop Scope

We would like to use our lab devices to build a small working path for functionality parallel to what our current batch tool does today:

- Deploy a Max Edge application update to a selected set of devices.
- Deploy a system utility or configuration update to a selected set of devices.
- Track job execution status per device.
- Report success, failure, timeout, and partial rollout state.
- Discuss how staged rollout, abort criteria, retry behavior, and rollback would work.
- Evaluate where AWS IoT Device Client is useful versus where we would need to integrate AWS patterns with **edged** or our existing edge services.
- Explore how device shadows could represent desired and reported state for application version, configuration version, health, connection state, and workload readiness.
- Discuss how fleet grouping, fleet indexing, and reporting could support operator workflows in Edge Manager.

Future State We Are Working Toward

Longer term, we want Edge Manager to provide a complete fleet management experience:

- Define a fleet as a subset of devices based on attributes, customer, site, application version, hardware profile, health, or other criteria.
- Declare a desired state for a fleet, including target application version, configuration version, utility package version, and relevant system settings.
- Reconcile devices toward that desired state with clear visibility into which devices are current, pending, failed, degraded, or intentionally held back.
- Support upgrade and downgrade workflows.
- Report device state, including online/offline status, connection health, currently running workloads, application version, configuration state, and green/yellow/red health.
- Notify users when updates are available for their devices, including options such as download only, download and install, defer, or schedule.
- Include human approval in the workflow when a device is running a critical workload that cannot be interrupted.

Desired Workshop Outputs

By the end of the workshop, we should have:

- A working lab demonstration of at least one update operation using AWS IoT capabilities.
- A recommended reference architecture for Day-2 fleet operations that fits our outbound-only security model.
- A clear view of which AWS components are appropriate for jobs, desired/reported state, fleet grouping, fleet reporting, and operator workflows.
- A gap list showing what AWS provides directly, what we should keep in Edge Manager, and what we would need to build ourselves.
- A pragmatic next-step plan for a follow-on proof of concept.

Request to AWS

Please support a focused workshop on Day-2 edge fleet operations using our lab devices and real update scenarios. The desired outcome is to determine whether AWS IoT Core can help us replace our current script-based batch tooling with a more scalable, state-aware, and operator-friendly fleet management foundation, and to identify the right AWS reference pattern for our security and operational constraints.